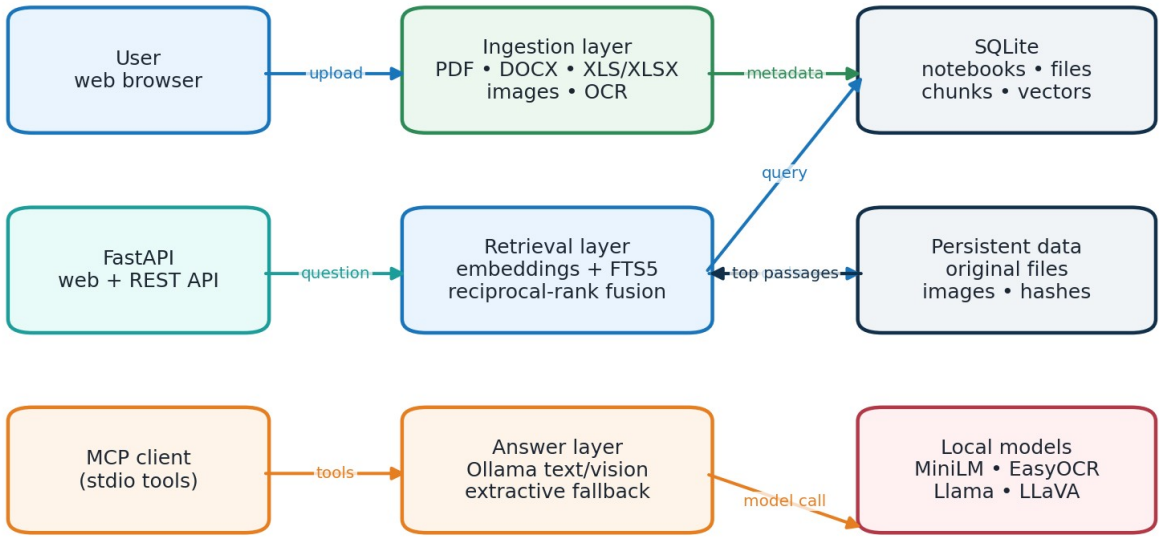
OFFLINE NOTEBOOK DOCUMENT QA

Secure Persistent Multimodal RAG for Isolated Laboratory Networks

15-page project report for guide review

End-to-end architecture



All services can run on one PC with no cloud API at runtime.

Figure 1. End-to-end system architecture

- Offline FastAPI service with native Python/Ollama and Docker deployment paths.
- PDF, DOCX, XLS/XLSX and image ingestion with page-aware extraction and OCR.
- Persistent SQLite catalog, original files, extracted images and local embeddings.
- Hybrid semantic plus SQLite FTS5 retrieval scoped to a selected notebook.
- Local text/vision answering with extractive fallback when Ollama is unavailable.
- MCP tools for folder ingestion, notebook operations and programmatic questions.

Prepared by: Cyril P. George | Repository: github.com/dhxvxn/URSC_RAG | 11 November 2026

1. Executive summary

Purpose, contribution and evidence boundary

This project is an offline document question-answering system for secure offices and laboratories. It turns mixed file collections into a persistent local knowledge base and answers questions from a selected notebook. The runtime can continue without internet after Python packages, OCR/embedding weights and Ollama models have been prepared.

The implementation emphasizes repeatability: a SHA-256 content hash prevents repeated extraction and embedding, while SQLite stores document metadata, chunks, vectors and notebook membership. Retrieval combines semantic similarity with FTS5 lexical search, then supplies only top-ranked evidence to a local text or vision model.

Implemented capability	Operational result
Persistent data	Documents, images, chunks and embeddings survive restart/rebuild.
Incremental folders	Later scans index only new or changed files.
Hybrid retrieval	Meaning and exact terms are both searchable.
Image evidence	OCR text is searchable; images can be returned or sent to LLaVA.
Notebook scope	A question is restricted to the selected collection.
Offline model path	No cloud API is required during normal runtime.

- The browser UI is the quickest demonstration path for a single PC.
- REST and Swagger expose the same operations for integration testing.
- MCP exposes tool calls to compatible AI clients without duplicating storage.
- The README documents Windows, Ubuntu, Docker, no-WSL and offline-bundle workflows.
- A standalone topic/entity graph is a planned extension, not a completed subsystem in the current baseline.
- The architecture is intentionally small enough to deploy on one server and inspect during a guide demonstration.
- Persistent source links allow a reviewer to validate where an answer came from.
- The same data model supports later network access without moving documents to client PCs.
- The model choice can be reduced for low-memory systems while the retrieval layer remains unchanged.
- The report treats offline operation as a deployment property that must be rehearsed, not assumed.
- The guide can evaluate the system using a representative notebook before approving a full archive import.

2. Problem statement and requirements

Why secure document QA needs a local architecture

Technical teams often search across mission reports, Word notes, spreadsheets, scanned pages and diagrams. Manual browsing is slow, and cloud AI services may be prohibited by confidentiality or network isolation. The project addresses these constraints by keeping indexing, retrieval, model calls and evidence storage on a controlled server.

Need	System response	Acceptance check
Avoid repeat uploads	SHA-256 deduplication	Second upload reports cached
Support mixed archives	PDF/DOCX/XLS/XLSX/images	Each format creates searchable evidence
Find concepts and codes	Embeddings + FTS5	Semantic and exact-term queries return hits
Use scans and figures	EasyOCR + image retention	OCR text and image URLs appear in results
Keep topics separate	Notebook membership	Queries filter by notebook ID
Operate offline	Local models and persistent data	Service works after disconnecting internet
Scale to workstations	API on server, Ollama private	Firewall exposes only approved port

- Primary demonstration: one Ubuntu or Windows PC with local browser access.
- Deployment target: one server with trusted lab workstations on an isolated subnet.
- Runtime data boundary: original documents, OCR results, images and vectors remain on the server.
- Quality boundary: answers must cite retrieved filenames/pages and admit when evidence is insufficient.
- Security boundary: authentication, HTTPS, audit logging and encrypted backups are required before wider production exposure.
- File boundary: supported extensions are accepted and unsupported files return an explicit skipped status.
- Evidence boundary: image sources remain available even when a vision model is not installed.
- Performance boundary: warm retrieval and model generation should be measured separately.
- Change boundary: a modified file is a new content version and is not silently merged with the old one.
- Operations boundary: model files, document data and application source are backed up as separate assets.
- Evaluation boundary: answer quality is judged against source citations as well as natural language fluency.

3. Architecture and component responsibilities

A modular pipeline that stays local

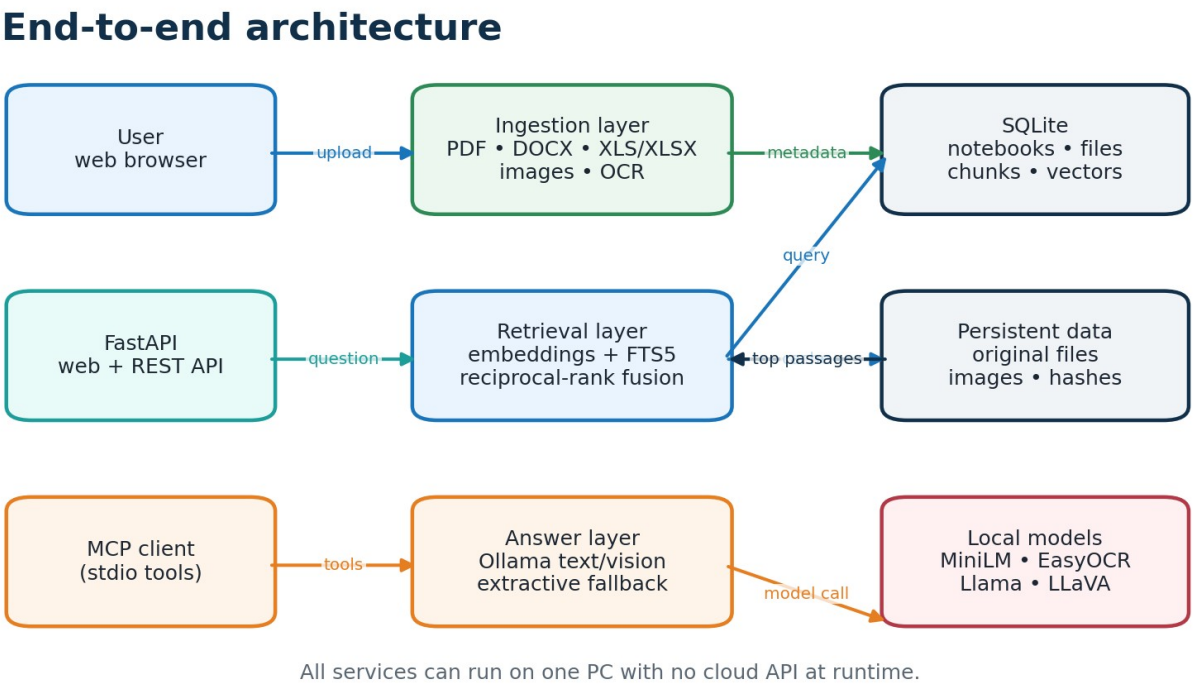


Figure 2. Application, persistence and model boundaries

Component	Responsibility
FastAPI	Serves the browser UI, health check, REST endpoints and source URLs.
Ingestion	Extracts text/tables/images, applies OCR and creates chunks.
Embedding layer	Creates normalized MiniLM vectors for chunks and queries.
SQLite	Stores notebooks, documents, memberships, chunks, vectors and FTS5 index.
Ollama	Runs local text/vision models and receives bounded retrieved evidence.
MCP adapter	Exposes list/create/ingest/ask operations over stdio.

- The web UI and MCP adapter call shared Python functions, so both interfaces use identical deduplication and retrieval rules.
- The embedding matrix for a notebook is loaded once and cached in process memory.
- The model service is contacted through localhost; it is not published to workstations.
- The API returns source metadata, image URLs and separate retrieval/answer timings.
- The health endpoint exposes notebook, document and chunk counts for operational checks.
- Persistent paths are configurable through DATA_DIR and model environment variables.
- The answer layer is replaceable because retrieval and generation are separate functions.
- The design keeps raw evidence available for human review instead of returning only a generated sentence.
- Each layer can be tested independently before the complete offline demonstration.

4. Ingestion, extraction and OCR

Converting heterogeneous files into evidence

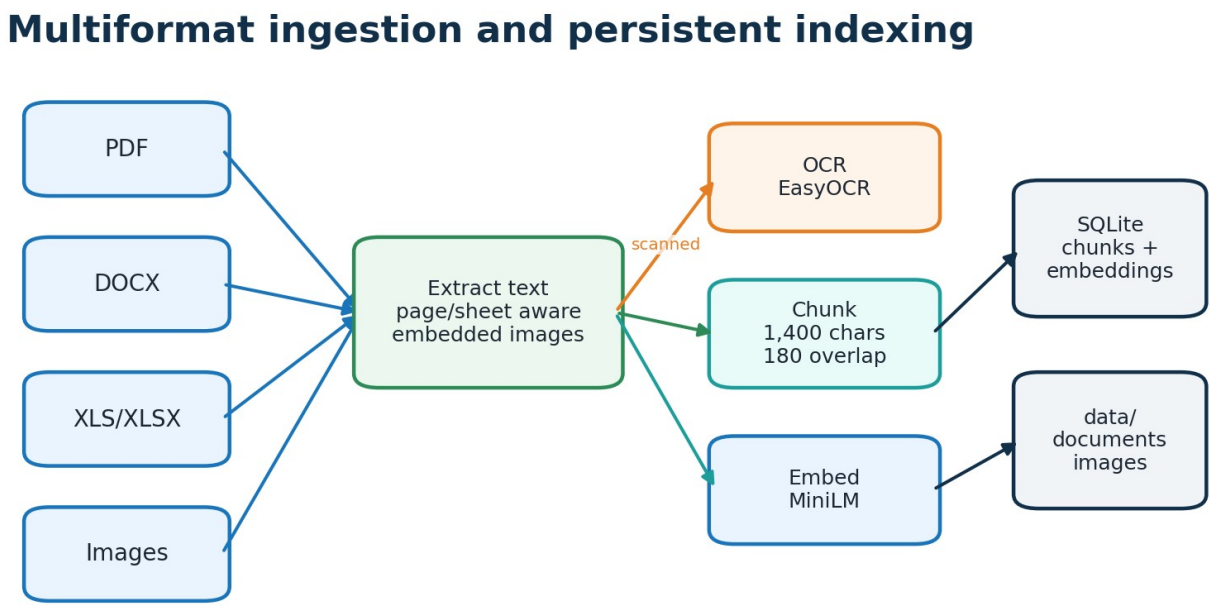


Figure 3. Multiformat ingestion path

The upload endpoint accepts multiple files and the browser supports selecting an entire folder. The MCP ingest_folder tool recursively walks a local directory. Unsupported extensions are reported as skipped. PDF text is read page by page; DOCX paragraphs and tables are flattened; Excel sheets become readable text blocks; embedded media is extracted and retained.

- Digital PDF pages use native text extraction for speed and accuracy.
- Image-only PDF pages are rendered and passed through EasyOCR.
- DOCX and XLSX media are extracted from their package contents.
- Raster images are stored and OCR text is indexed when available.
- Page and sheet numbers remain attached to chunks for traceable citations.
- OCR weights are prepared before isolation and download_enabled is disabled at runtime.
- A file with no readable text is reported as an indexing error rather than silently accepted.
- Filenames are sanitized before storage so folder paths cannot escape the data directory.
- Each upload produces a per-file status, making partial folder failures visible to the user.
- Sheet names are retained in the extracted text to preserve spreadsheet context.
- Extracted images remain connected to their document and page for later source display.
- The stored original is never replaced by OCR output; OCR is an additional evidence channel.
- A later folder scan can safely include unchanged files because the hash check is idempotent.

5. Embeddings, indexing and hybrid retrieval

Fast search for concepts, identifiers and technical language

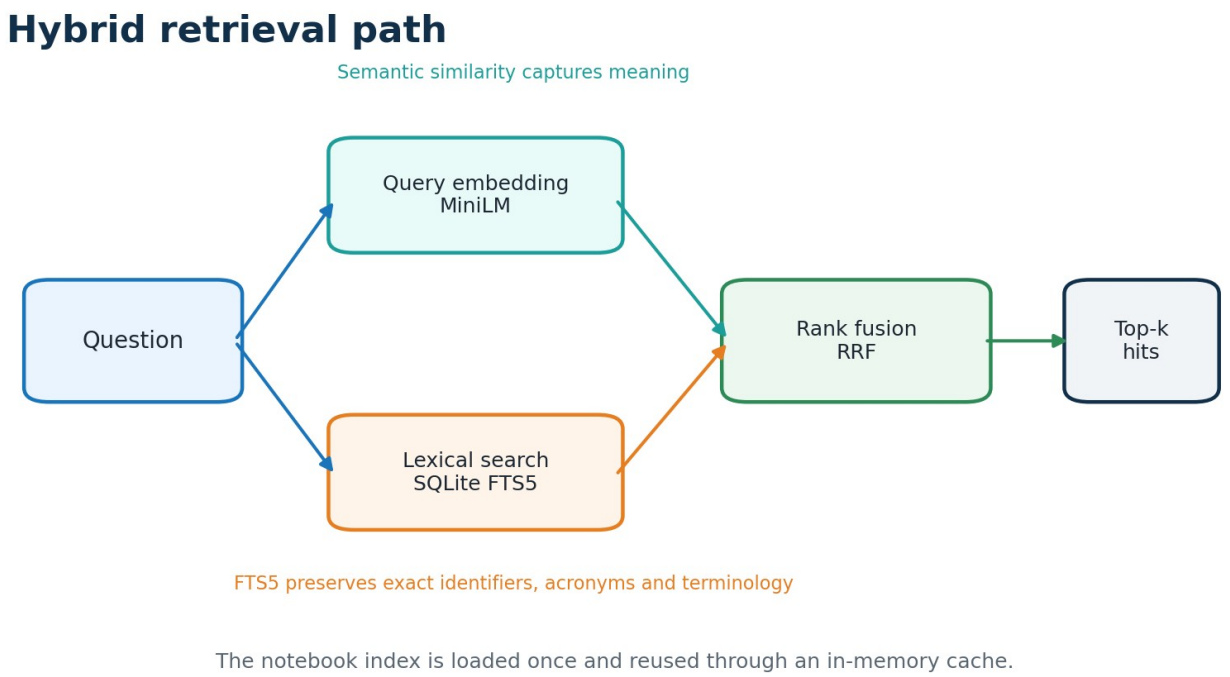


Figure 4. Semantic and lexical retrieval are fused

Text is normalized and divided into chunks of approximately 1,400 characters with 180 characters of overlap. The all-MiniLM-L6-v2 model encodes chunks into normalized float32 vectors. At query time, cosine similarity is a matrix multiplication over the selected notebook. FTS5 supplies exact-term candidates for codes, acronyms and identifiers.

- Candidate limits prevent the search phase from scanning unnecessary results.
- Reciprocal-rank fusion combines semantic and lexical rankings without collection-specific thresholds.
- The notebook cache avoids repeated BLOB decoding after the first query.
- Cache invalidation occurs after ingestion so new content becomes visible immediately.
- Top-k limits and bounded context protect answer latency and model memory.
- Ollama keep-alive reduces repeated model-load delays between questions.
- The response exposes retrieval_ms separately from answer_ms for honest benchmarking.
- Normalized vectors make the dot product equivalent to cosine similarity.
- Lexical terms are limited and sanitized before being sent to FTS5.
- Semantic candidates and lexical candidates are combined by rank, not by unstable raw scores.
- Top-k is bounded by the API request model to avoid unbounded prompt growth.
- The retrieved metadata retains filename, document ID, page and chunk text for citations.
- A notebook with no indexed content returns a clear no-match result instead of an empty model prompt.

6. Answer generation and image-aware QA

Grounded local answers with fallback behavior

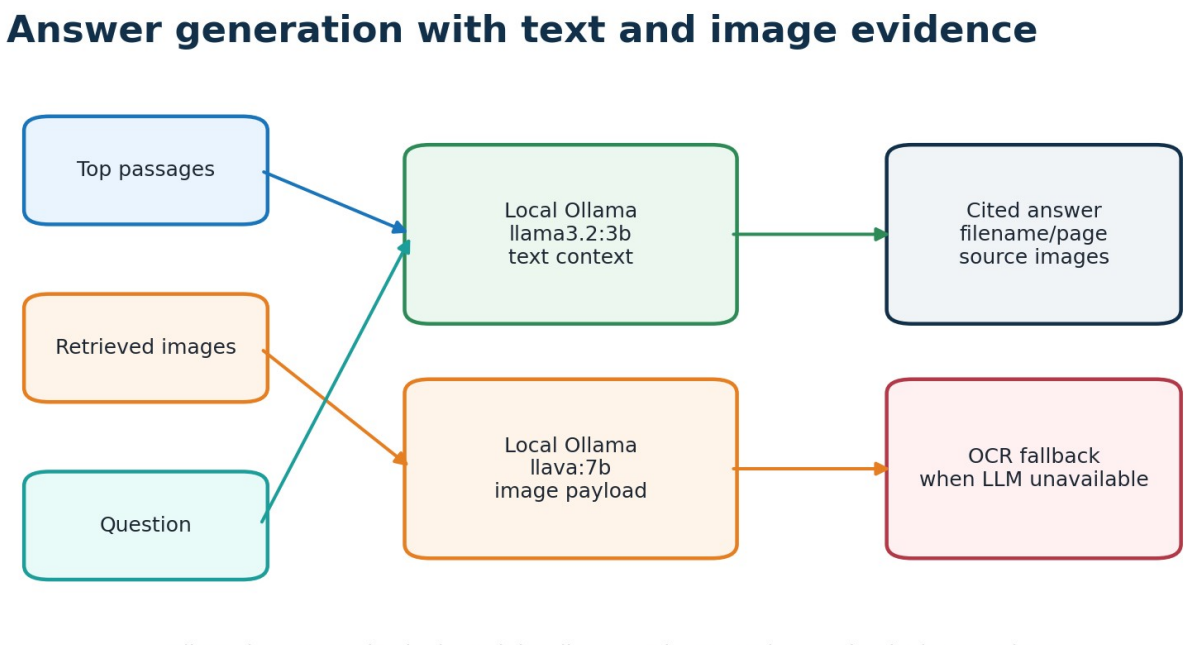


Figure 5. Text and visual evidence are kept inside the local trust boundary

The answer prompt instructs Ollama to use only supplied notebook sources, cite filename and page where available, and say when the sources do not contain an answer. The default text model is llama3.2:3b. When retrieved images exist, llava:7b receives a bounded image payload and the same evidence context.

- Text-only questions use the smaller text model and usually require less memory.
- Questions involving diagrams can use the optional vision model.
- Image OCR remains useful even when a vision model is not installed.
- The API returns image URLs alongside source passages for human review.
- If Ollama is stopped or times out, extractive_fallback returns the strongest passages.
- The fallback keeps search useful during startup, low-memory operation or model maintenance.
- The bounded prompt reduces irrelevant context and supports predictable latency.
- The system prompt identifies the assistant as an offline document research assistant.
- Source filenames and page numbers are included in the context before the question.
- Only images associated with retrieved documents are considered for a vision request.
- MAX_VISION_IMAGES limits the visual payload and protects memory on small PCs.
- HTTP failures are caught so the fallback remains available during model outages.
- Human reviewers can inspect returned images rather than trusting an opaque visual answer.

7. Persistence, deduplication and notebooks

One-time indexing with topic-scoped collections

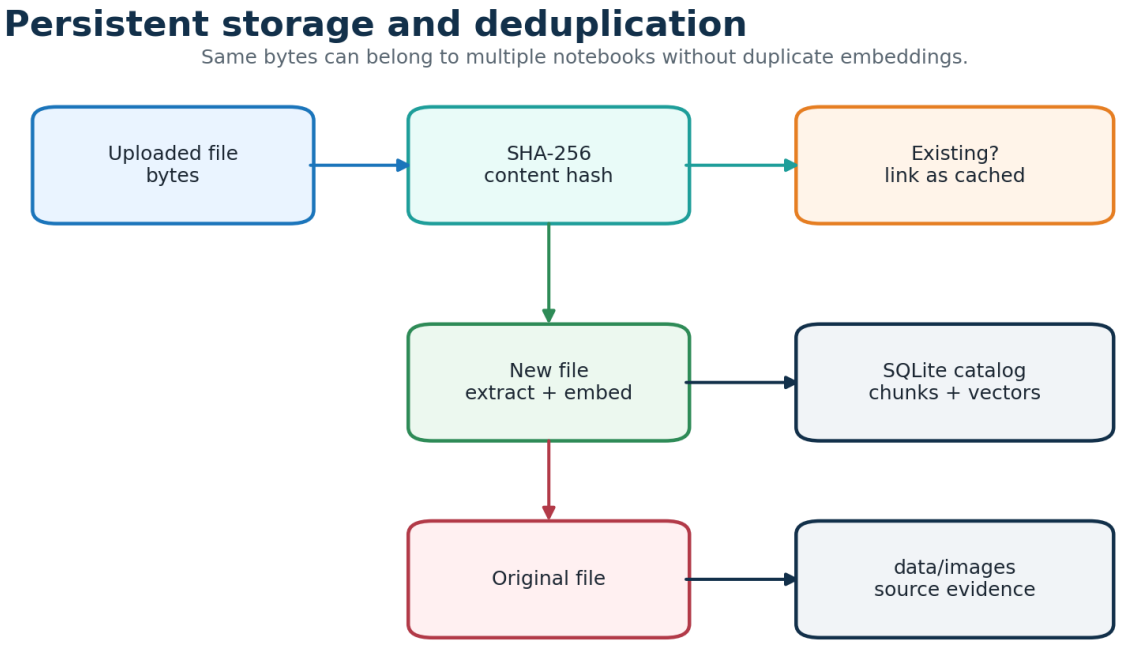


Figure 6. Content hashing controls repeat work

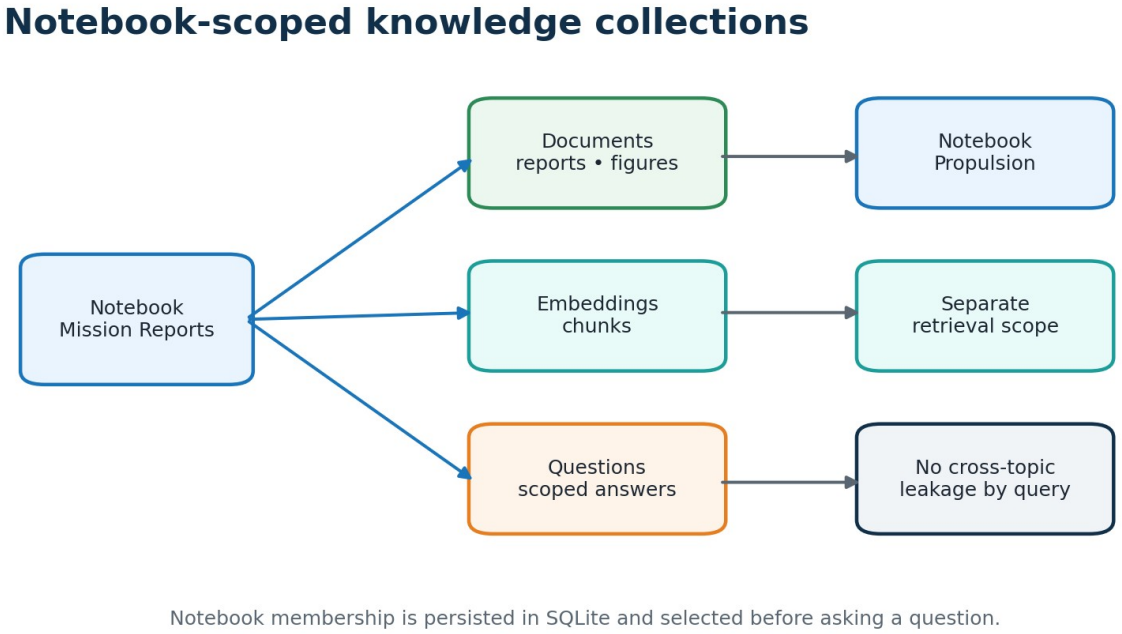


Figure 7. Notebook membership controls retrieval scope

- A SHA-256 digest identifies file bytes independently of filename or folder location.
- Repeated bytes are linked to another notebook without new extraction or embeddings.
- Changed bytes receive a new digest and are indexed as a new version.
- The data directory contains the SQLite catalog, originals and extracted images.
- Docker uses named volumes; native deployment uses data/ and models/ directories.
- Notebook membership allows one document to support several project collections.
- Queries join chunks through notebook membership before ranking, reducing unrelated hits.

8. Knowledge graph readiness and roadmap

From document links to topics, entities and relations

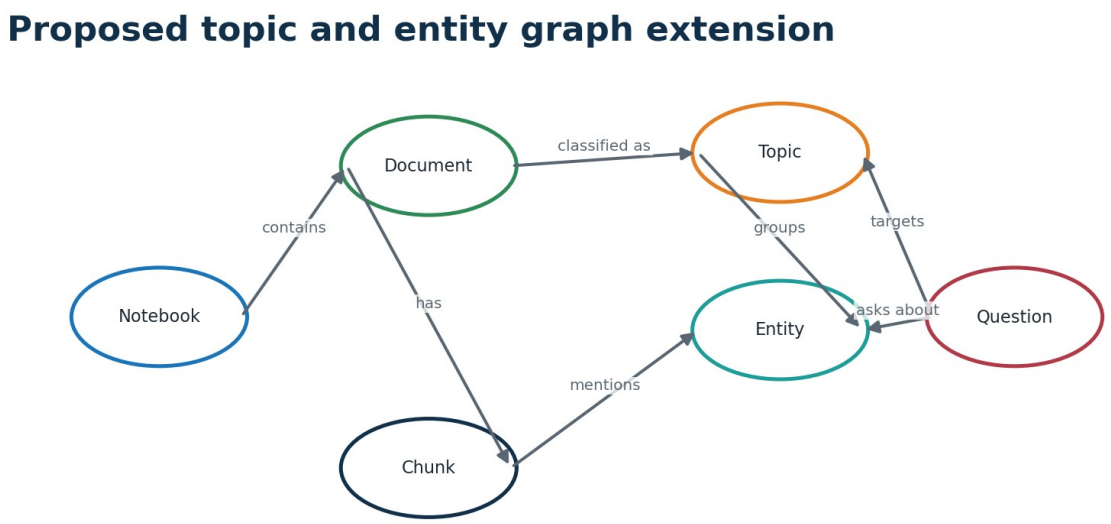


Figure 8. Proposed graph layer over the current index

The current baseline already persists graph-like links: notebooks contain documents, documents produce page/sheet-aware chunks, and chunks can be associated with extracted images. A separate topic/entity graph with typed relations is not yet implemented; it should be added without replacing the fast hybrid index.

Future graph element	Example	Value
Topic	Trajectory prediction	Topic navigation and related-topic expansion
Entity	LEO, sensor, mission	Exact entity filtering and disambiguation
Relation	mission uses sensor	Multi-hop context across documents
Mention	chunk 184 mentions LEO	Traceable link back to evidence

- Run entity/topic extraction asynchronously after basic indexing completes.
- Persist graph nodes and edges in SQLite or an approved local Neo4j deployment.
- Use graph links for candidate expansion and a small ranking bonus.
- Keep dense and FTS5 retrieval as the fallback for robustness and speed.
- Evaluate graph precision on the lab's real terminology before enabling automatic answer expansion.
- Store every graph mention with a document and chunk identifier for explainability.
- Use topic labels to organize notebooks without copying the underlying documents.
- Resolve aliases such as abbreviations and alternate spacecraft names before creating edges.
- Reject low-confidence relations rather than adding noisy graph connections automatically.
- Expose graph neighbors as additional candidates, then let source ranking decide the final context.
- Keep the graph optional so text retrieval remains available if graph extraction fails.

9. API, web interface and MCP

Browser, automation and agent integration

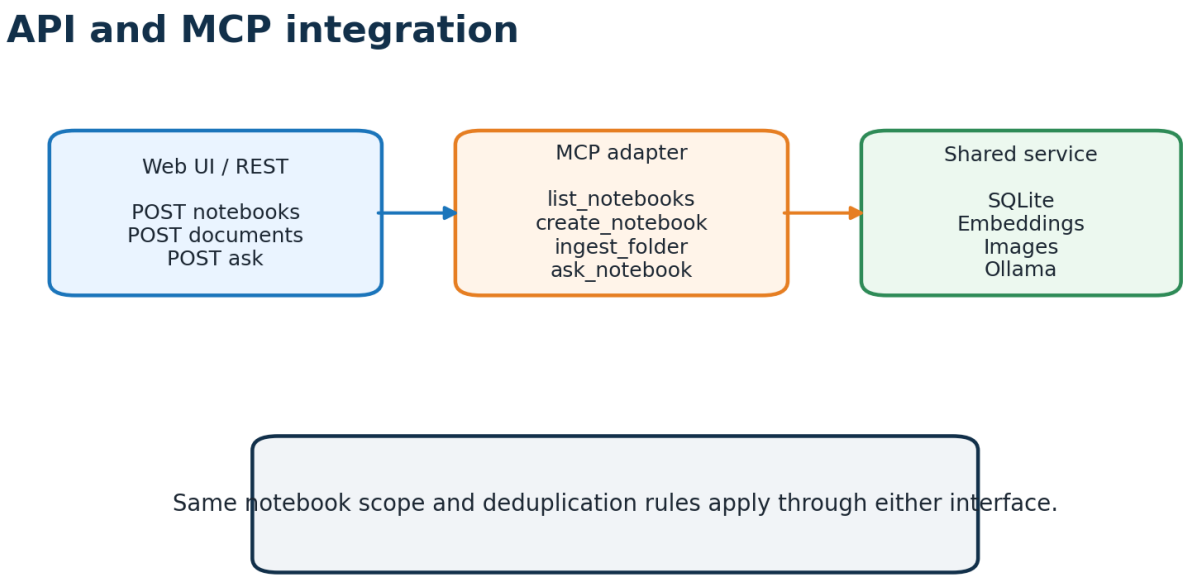


Figure 9. REST and MCP share the same data and retrieval services

Interface	Operation	Use
REST	GET /health	Counts, service status and retrieval mode
REST	POST /api/notebooks	Create a notebook
REST	POST …/documents	Index uploaded files
REST	POST …/ask	Retrieve, answer and return sources/images
MCP	list/create_notebook	Manage collections from an AI client
MCP	ingest_folder	Recursively index local folders
MCP	ask_notebook	Ask against one notebook

- MCP is an integration interface, not a second database or a replacement for the web UI.
- An MCP client starts mcp_server.py over stdio and calls structured tools.
- The same content hashes prevent duplicate work through either interface.
- The same notebook ID keeps answers scoped to the selected collection.
- Trusted local clients should be used because the current adapter has no independent user login.
- The /health endpoint is suitable for service monitors and deployment smoke tests.
- REST clients can upload multiple files using multipart form data.
- The ask response includes sources, images, retrieval mode and timing fields.
- MCP lets an agent ingest a folder without reproducing the HTTP upload loop.
- Both interfaces can be used while the host is disconnected from the public internet.
- A future authenticated gateway should apply the same notebook permissions to REST and MCP.

10. Deployment architecture

Single-PC demonstration to protected lab server

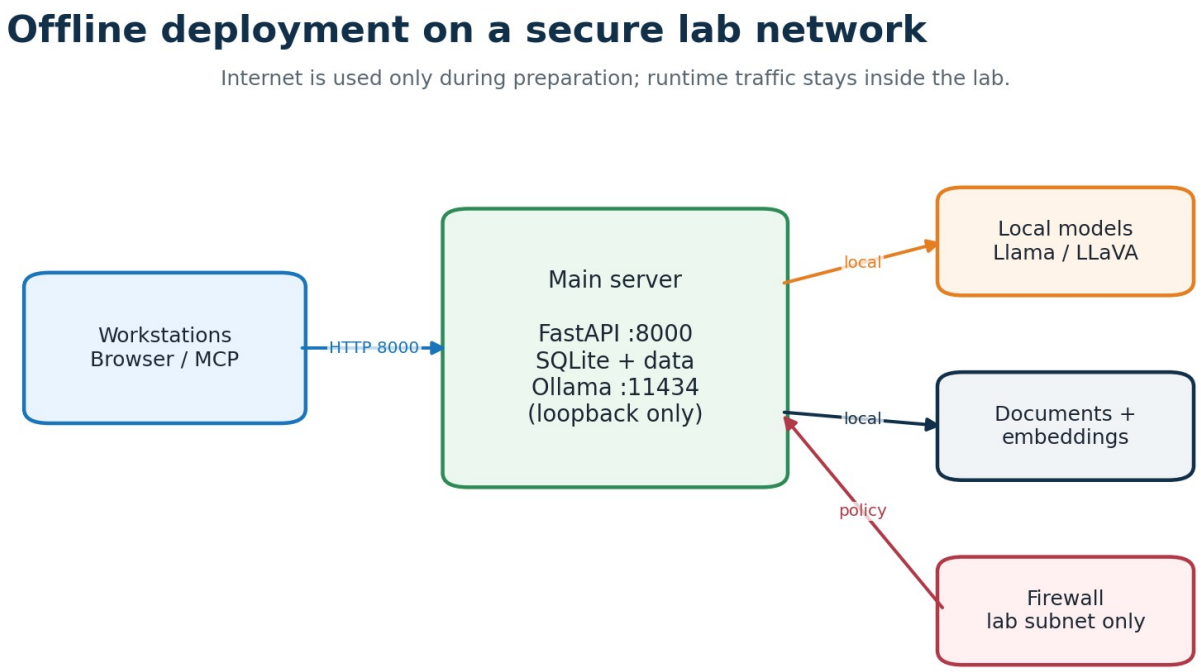


Figure 10. Recommended lab-network trust boundary

Mode	Preparation	Runtime
Native	Python wheels, model directories and Ollama installer	FastAPI + local Ollama processes
Docker	Build image and pull models while connected	docker compose up -d with named volumes
Lab server	Prepare once and restrict firewall	Workstations use API port only

- The default API binding is loopback-only for a safe one-PC demonstration.
- For a lab subnet, bind port 8000 to the server interface and keep Ollama on 127.0.0.1:11434.
- Allow only the approved workstation subnet through the host firewall.
- Clients access http://SERVER_IP:8000; they do not access the model port or data directory.
- Add HTTPS, authentication and authorization before exposing the service beyond a trusted subnet.
- Back up application data and Ollama models before replacing the server.
- Use a static server address or approved internal DNS for workstation access.
- Keep the API binding at 127.0.0.1 until the single-PC demonstration is complete.
- Verify the firewall rule from an approved workstation and from an unapproved address.
- Do not mount the document volume into workstation containers or shares unnecessarily.
- Separate model storage from document storage so each can be backed up and restored independently.
- Document the server IP, port, model versions and restart procedure for operators.

11. Offline installation and operations

Preparation once; repeatable use without internet

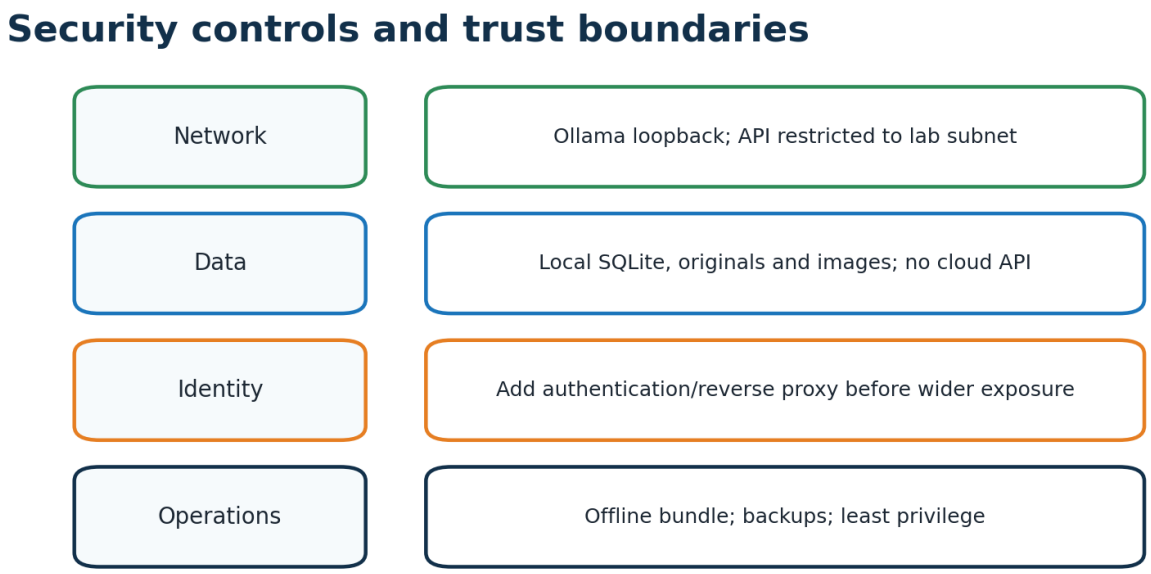
The preparation machine downloads the repository, Python dependencies, embedding/OCR weights, Ollama installers and language models. The target receives an operating-system-specific bundle. A virtual environment is rebuilt on the target; it should not be copied between Windows and Ubuntu.

Step	Windows native	Ubuntu native
Prepare	.\scripts\setup_native.ps1	./scripts/setup_native.sh
Start Ollama	.\scripts\run_ollama.ps1	./scripts/run_ollama.sh
Models	ollama pull llama3.2:3b; llava:7b	same commands
Start API	.\scripts\run_native.ps1	./scripts/run_native.sh
Check	curl.exe .../health	curl .../health

- Copy source files, offline wheels, models/ and optionally data/ to approved removable media or an internal share.
- Run setup with the offline bundle so pip never contacts the internet.
- Keep Ollama and FastAPI running in separate terminals or services.
- Create a notebook, select a folder and index only after /health reports success.
- Disconnect internet after models and dependencies have been verified.
- Do not run pip install, ollama pull or docker compose build on the isolated target.
- Never use docker compose down -v unless deleting the catalog and embeddings is intentional.
- Prepare wheels for the same operating system, Python version and CPU architecture as the target.
- Copy model directories with their directory structure so offline paths resolve without downloads.
- Run curl against /health before indexing a large folder.
- Use the cached status returned by the upload API to confirm that repeat files were reused.
- Back up data/ before changing model versions or moving the deployment.
- Keep a written inventory of installers, wheels, model files and checksums for the offline handover.

12. Security, privacy and controls

Protecting sensitive office documents



The current application is suitable for a trusted lab prototype; production authentication and audit logging remain required.

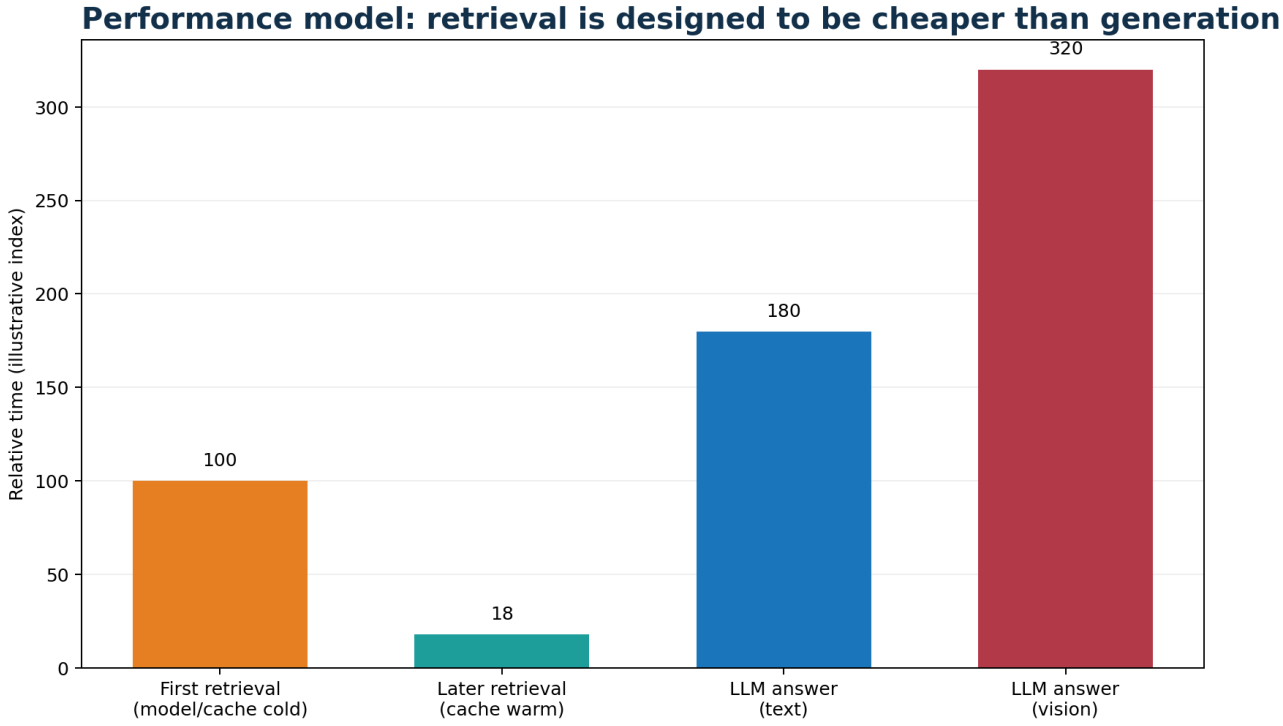
Figure 11. Security controls by trust boundary

Risk	Current control	Production improvement
Cloud exposure	No cloud API at runtime	Approved model registry and checksums
Ollama exposure	Loopback binding	Keep port private and firewall verified
API access	Loopback default and subnet guidance	Login, roles and HTTPS
Backup leakage	Local volume guidance	Encryption and access-controlled restore
Prompt injection	Source-only prompt instruction	Input/output review and policy tests
Resource abuse	Top-k/context limits	Upload quotas and monitoring

- The data directory contains originals and must be protected like the source documents.
- Backups should be created while services are stopped for a consistent snapshot.
- Restrict file permissions so only the application account and approved administrators can read storage.
- Use a reverse proxy for authentication, TLS termination and request logging.
- Record model versions and checksums so offline deployments can be reproduced.
- Review prompt, retrieved sources and final answer when testing sensitive workflows.
- Apply least-privilege permissions to the service account and model directories.
- Treat uploaded files and extracted images as sensitive records in backup and incident procedures.
- Test the service with oversized, malformed and unsupported files before lab rollout.
- Record who can create notebooks and which collections each user may query.
- Use an offline patch process with approved packages and documented change review.
- Plan data retention and secure deletion policies for documents no longer required.

13. Performance, validation and limitations

How to demonstrate quality honestly



Illustrative engineering model, not a benchmark; measure on the target PC with its corpus and model quantization.

Figure 12. Illustrative timing model; measure on target hardware

Test	Measure	Expected evidence
Compilation	py_compile main.py mcp_server.py	No syntax errors
Indexing	Files/minute by format	New files indexed once
Deduplication	Repeat same upload	cached; no new chunks
Warm retrieval	retrieval_ms after cache warm-up	Low stable search latency
Answering	answer_ms for text/vision	Model choice matches hardware
Offline rehearsal	Disconnect after preparation	Health, ingest and ask still work
Growth	1k/10k/100k chunks	Know when ANN index is needed

- Retrieval is designed to be cheaper than generation because vectors are cached and context is bounded.
- First use includes model loading and may be slower than warm queries.
- Vision answers require more RAM and storage than text-only answers.
- SQLite is appropriate for an initial single-server corpus; larger collections may need an ANN index.
- The current baseline has no enterprise authentication, formal audit trail or completed entity graph.
- A target-PC benchmark with real documents is required before making a hard latency guarantee.
- Record cold-start and warm-cache timings separately because they represent different user experiences.
- Measure index throughput for digital PDFs, scans, spreadsheets and image-heavy folders.
- Compare text-only and vision answers on the same evidence set.
- Track cache hit rate and database size as notebooks grow.
- Use source citation accuracy as a quality metric in addition to response time.
- Move to an ANN index or background workers only when measured growth justifies the complexity.

14. Conclusion and recommended next steps

A practical foundation for secure offline document intelligence

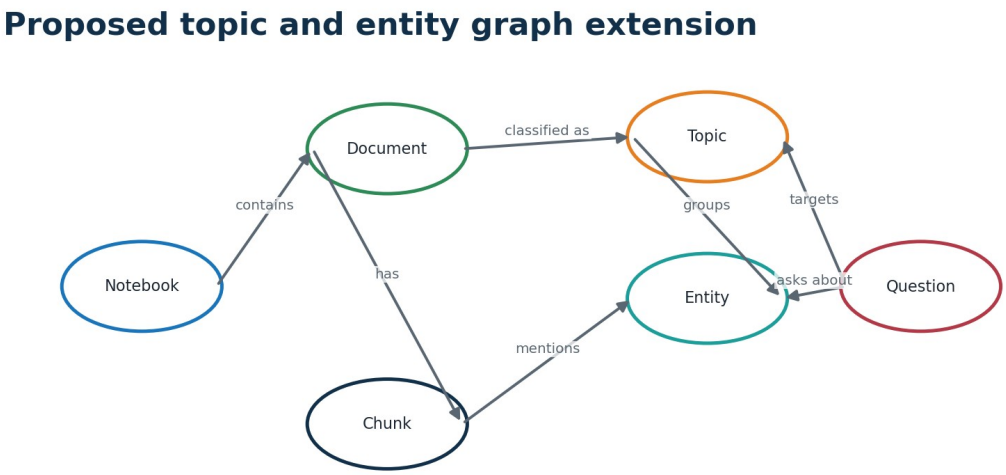


Figure 13. The graph roadmap builds on the persistent index

Offline Notebook Document QA provides a practical first deployment for a secure office: it accepts common documents and folders, embeds them once, preserves source evidence, retrieves quickly inside a selected notebook, returns relevant images and uses local Ollama models for grounded answers. The same store supports a browser, REST clients and MCP tools.

- Demonstrate the native deployment on one Ubuntu PC before adding network clients.
- Measure indexing time, warm retrieval time and answer time using the guide's sample corpus.
- Add authentication, HTTPS, role-based notebook permissions and audit logging.
- Implement asynchronous entity/topic extraction and persist a local knowledge graph.
- Add encrypted backup/restore and verify the offline bundle on a clean target machine.
- Introduce an ANN vector index only when corpus size or measured latency requires it.
- Document model versions, hardware requirements and acceptance thresholds for the lab.
- Start with one approved notebook and a small representative sample before importing the full archive.
- Review the graph schema with domain experts before extracting relationships at scale.
- Use the MCP interface when a guide wants to demonstrate conversational or automated workflows.
- Keep the README and this report synchronized with the deployed commit.
- Record the final server address, backup location and operator contact in the handover notes.

Repository: github.com/dhxvxn/URSC_RAG. The README contains native Windows, native Ubuntu, Docker, no-WSL, offline-bundle, lab-network and troubleshooting instructions. This 15-page report describes the repository state inspected on 11 November 2026 and separates implemented behavior from proposed extensions.